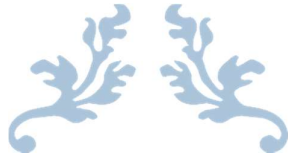


UNIVERSITY OF MESSINA



DATA CLASSIFICATION

Data Mining and Analysis



Prof. Gennario Tartarisco

Prof. Simona Campisi

Students:

- Aditi Dole (533674)
- Jignesh Goyani (540106)
- Meet Tagadiya (540285)

Data Exploration:

- 425 observations (142 subjects x 3 conditions: baseline, gratitude and neutral). Indeed, 5 subjects were excluded because the ECG signal was missing.
- 3 labels indicating the different conditions from which the features were extracted. Specifically:
 - Baseline condition = -1
 - Gratitude = 319
 - Neutral State = 109
- Features extracted from the RR signal, which is the time elapsed between two successive R waves of the QRS signal on the electrocardiogram, including time and frequency domain features, as well as non-linear features.

Objective:

The primary goal of this analysis is to perform classification on the dataset using feature selection techniques. The features will be scrutinised and selected based on various approaches, and subsequently, different classification models will be employed to assess their effectiveness.

1. Data Normalisation:

A. The given data frame after import was checked for anomalies:

```
#import data and explore data
df = pd.read_csv("Dataset_Study3.csv")
#df.head()
print("info " , df.info())
print("Desc" ,df.describe())
print("Is null ",df.isnull().value_counts())
print("is duplicate: ",df.duplicated().value_counts())
```

✓ 0.1s

B. The target variable was split.

C. Data was then scaled using a standard scaler. The standard scaler standardizes features by removing the mean and scales to a common variance. This step is essential to machine learning estimators.

D. Data Visualisation: The data was visualised with various plots such as boxplot and scatterplot. Visualising data gives an idea about the distribution of the data. The boxplot allows us to identify the outliers.

E. Outliers: The outliers in this particular data are very few. Hence they were ignored as they do not affect the data overall.

2. Feature selection:

Feature selection is the process of selecting a subset of relevant features (variables, predictors) for use in model construction. It can help to simplify your models, reduce training time, avoid overfitting, and improve accuracy.

A. Filter Methods:

- 1) Correlation: The Correlation Matrix enables to identification of the features and their correlation. The highly correlated features are dropped due to their similarity. High correlation reduces the performance of the model.
- 2) Variance: Features with variance lower than the specified threshold are removed.
- 3) Mutual info: Mutual info classif was used to measure the dependency between two variables.

B. Univariate feature selection:

- 4) K- best with chi score: The features are sorted by the chi score (association between two categorical variables). However, no result as the data consists of negative values.

C. Wrapper methods:

- 5) RFE: Recursively removes the least important features based on a model's performance. RFE had Logistic regression as its base estimator.

D. Sequential feature selection: Based on Accuracy and Cross-validation with 5 fold:

- 6) SFS_forward: Adding the most significant features based on accuracy.
- 7) SFS_backward: Removes least significant in each iteration.

E. Embedded methods:

- 8) LASSO: yielded no results.
- 9) Decision Tree: Feature importance is considered.

F. Dimensionality reduction:

- 10) PCA: Reduces dimensionality by transforming features into uncorrelated principal components.

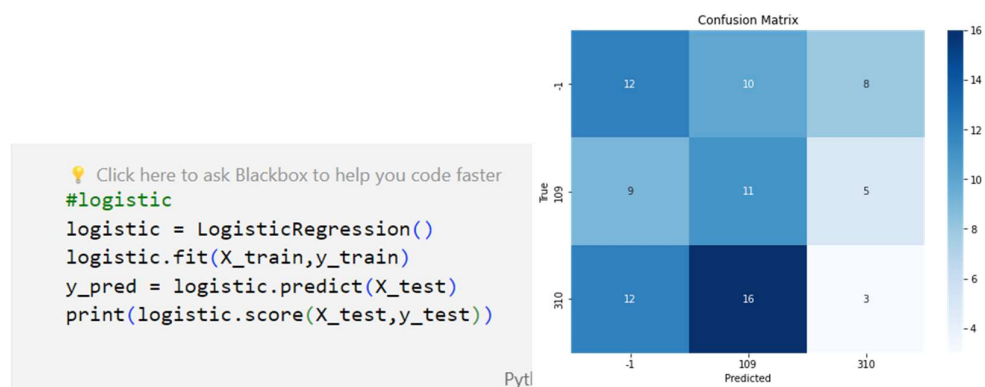
G. Other:

- 11) Random forest.

3. Classification Models :

The target variable: Label consists of 3 categories. Thus this is a multiclass classification. The data is then split with a test size of 20% of the total. The rest is training data. The models are trained on the latter allowing the model to learn. Then the prediction is tested on the former.

- 1) Logistic Regression: Computes the probability of an instance belonging to a particular class. In this case, the range is (0,1,2):

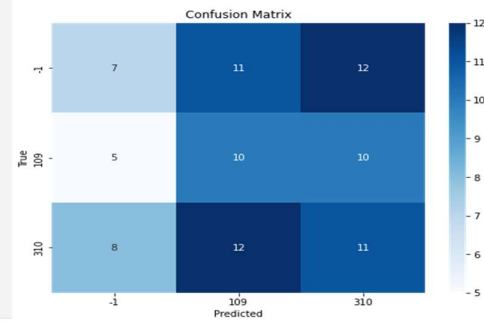


- 2) Decision tree: Hierarchical tree-like structure of decision nodes. Each node represents a decision based on a specific feature, leading to subsequent nodes until a leaf node with a predicted outcome is reached.

Click here to ask Blackbox to help you code faster

```
#dtree
dtree = DecisionTreeClassifier()
dtree.fit(X_train,y_train)
y_pred = dtree.predict(X_test)
print(dtree.score(X_test,y_test))
```

✓ 0.0s



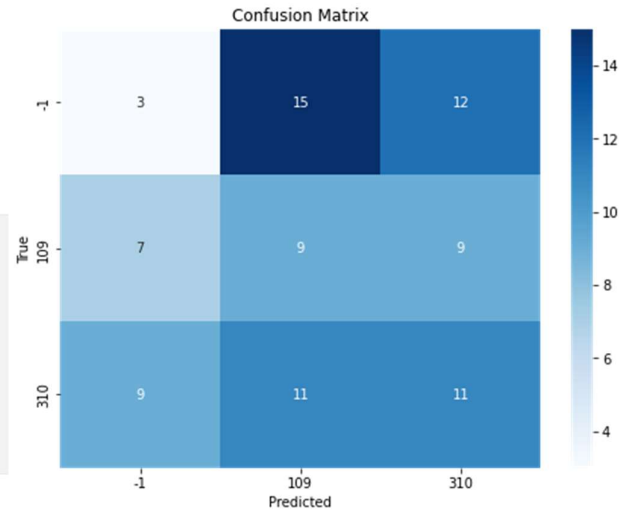
- 3) Random forest: Constructs multiple decision trees during training and merges their predictions. Each tree is trained on a random subset of the training data and a random subset of features.

Click here to ask Blackbox to help you code faster

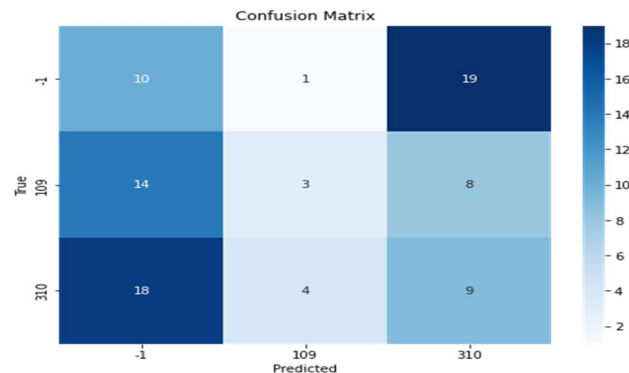
```
#random forest
rf_model = RandomForestClassifier()
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
print(rf_model.score(X_test,y_test))
```

✓ 0.3s

0.26744186046511625



- 4) SVM: Separates data into classes by finding the hyperplane that maximally separates the classes. Can use different kernel functions to handle non-linear decision boundaries.



💡 Click here to ask Blackbox to help you code faster

```
#SVM
svm_classifier = svm.SVC(kernel='linear', C=1.0, random_state=42)
svm_classifier.fit(X_train,y_train)
y_pred = svm_classifier.predict(X_test)
print(svm_classifier.score(X_test,y_test))

svm_classifier = svm.SVC(kernel='poly', C=1.0, random_state=42)
svm_classifier.fit(X_train,y_train)
y_pred = svm_classifier.predict(X_test)
print(svm_classifier.score(X_test,y_test))

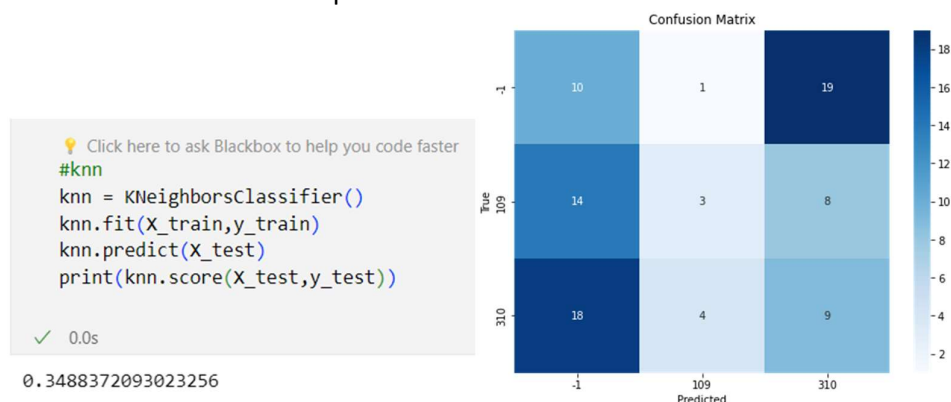
svm_classifier = svm.SVC(kernel='rbf', C=1.0, random_state=42)
svm_classifier.fit(X_train,y_train)
y_pred = svm_classifier.predict(X_test)
print(svm_classifier.score(X_test,y_test))

svm_classifier = svm.SVC(kernel='sigmoid', C=1.0, random_state=42)
svm_classifier.fit(X_train,y_train)
svm_classifier.predict(X_test)
print(svm_classifier.score(X_test,y_test))
```

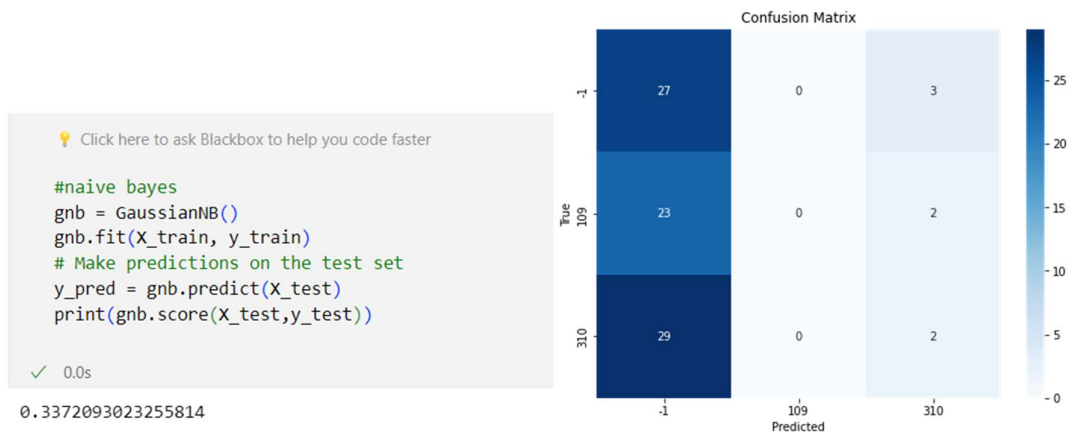
✓ 0.0s

0.313953488372093
0.3023255813953488
0.2558139534883721
0.313953488372093

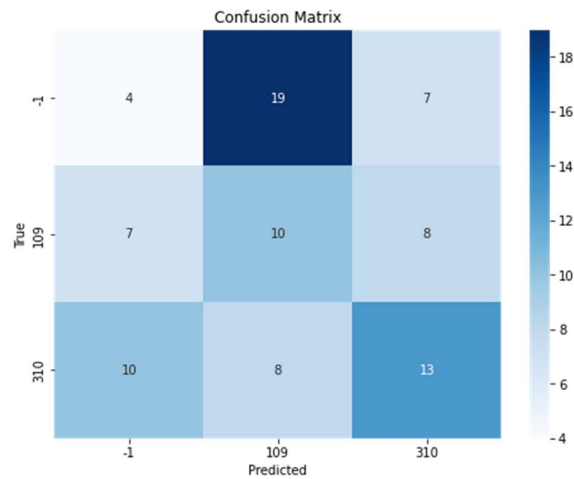
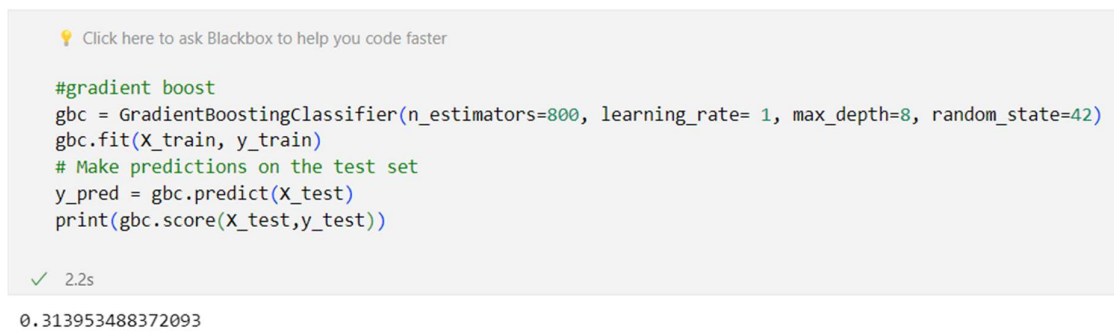
- 5) KNN: Classifies a new instance based on the majority class of its k nearest neighbours. The choice of k is a critical parameter.



- 6) Naïve Bayes: Assumes that features are conditionally independent given the class label. Uses Bayes' theorem to calculate the probability of a class given the observed features.



- 7) Gradient boosting: Builds a series of weak learners (typically decision trees) sequentially, with each tree correcting the errors of the previous one. The final prediction is a weighted sum of the predictions of all trees.



4. [Models and their accuracy:](#)

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.430233	0.430140	0.430233	0.420170
1	Decision Tree	0.360465	0.364007	0.360465	0.359726
2	K-Nearest Neighbors	0.302326	0.303791	0.302326	0.292782
3	Naive Bayes	0.360465	0.392715	0.360465	0.358087
4	Gradient Boost	0.290698	0.288521	0.290698	0.288102
5	Random Forest	0.348837	0.330566	0.348837	0.336893

Conclusion:

In this analysis of a dataset with 425 observations across 3 categories, we explored features derived from the RR signal's time, frequency, and non-linear aspects. Our objective was classification through meticulous feature selection and testing various models.

After standardizing data, diverse feature selection methods were employed, including filter methods, univariate selection, wrapper methods, and dimensionality reduction. Classification models such as Logistic Regression, Decision Tree, Random Forest, SVM, KNN, Naive Bayes, and Gradient Boosting were tested

Validation via cross-validation revealed accuracy ranging from 25% to 45% highlighting dataset intricacies. Precision, recall, and F1-score were considered for a comprehensive evaluation.

While the results provide valuable insights, acknowledging potential unknown errors is crucial. Further investigation is recommended for refining models and improving predictive capabilities. This analysis sets the groundwork for iterative enhancements and underscores the continuous nature of data analysis.